# EE587 - MILESTONE 2 - SOBEL EDGE DETECTION ON BASYS3 FPGA

HARISHRAJ T.

E/21/171

SEMESTER 06

09/08/2026

# EE587 - Milestone 2 - Sobel Edge Detection on Basys3 FPGA

Milestone 2 Report

HARISHRAJ T. E/21/171 Semester 06 09/08/2026

## 1. Introduction

This report documents a real-time Sobel edge detection system implemented in Verilog for the Digilent Basys3 FPGA board. The design reads a static 8-bit grayscale image from an on-chip Block RAM (BRAM) image store, streams every pixel through a 3×3 Sobel gradient pipeline, caches the resulting edge map in a second BRAM frame buffer, and continuously displays that buffer on a standard VGA monitor at 640×480 (60 Hz).

The design is organised around a “process-once, then display” model: the entire source image is pushed through the Sobel core once, immediately after reset, and the resulting edge map is cached. The VGA output stage then simply scans out the cached frame buffer every refresh, so the Sobel computation itself does not have to keep pace with the video raster in real time.

### 1.1 Objectives

- Store a static grayscale image in on-chip Block RAM and make it available for pixel-streaming.
- Implement a fully pipelined 3×3 Sobel gradient operator that processes the image as a single continuous stream.
- Buffer the computed edge map in a second BRAM so it can be displayed independently of the compute pipeline's timing.
- Generate standard VGA 640×480 @ 60 Hz timing and centre the processed image within the visible frame.
- Verify the compute path in simulation, independent of VGA timing, by dumping the frame buffer to a memory file for offline image inspection.

## 2. System Architecture Overview

The top-level module top_sobel_vga instantiates and wires together five sub-modules. Unlike a fully streaming design where every stage operates in lock-step with the VGA pixel clock, this design decouples computation from display using an intermediate frame buffer, as summarized below.

### 2.1 Module Hierarchy

```
top_sobel_vga (top_sobel_vga.v)
        pixel_clk_en   (pixel_clk_en.v)
                1-in-4 enable strobe on the 100 MHz domain (25 MHz equivalent rate)
        image_rom      (image_rom.v)
                Source image BRAM, 8-bit grayscale, $readmemh-initialised
        sobel_core     (sobel_core.v)
                Row buffers + 3x3 gradient pipeline (Gx, Gy, magnitude, saturation)
        output_bram    (output_bram.v)
                Frame buffer holding the computed edge map (write side: sobel_core,
                read side: VGA scan-out)
        vga_timing     (vga_timing.v)
                640x480 @ 60 Hz H/V counters, hsync/vsync, video_on
```

## 2.2 Data Flow

There are two independent data paths that share the same 100 MHz clock domain but run at different rates:

- Compute path : on release of reset, an address counter (addr_cnt) free-runs through every pixel of the source image, once, at the full 100 MHz clock rate. Each pixel is pushed into sobel_core, which streams out the corresponding gradient magnitude. The result is written into output_bram.
- Display path : the VGA timing generator runs continuously, gated by a pixel_en strobe that fires 25 MHz effective rate so the whole design stays in a single 100 MHz clock domain , there is no separate divided clock net or BUFG for pixel timing. Once compute_done is asserted, the active display window reads back from output_bram and drives the VGA colour outputs.

## 2.3 Image Preparation Toolchain

‘image_to_mem.py’ takes any input image, converts it to grayscale, scales it down (preserving aspect ratio, capped at ‘max_width’ /‘max_height’, default 200×300) using Lanczos resampling, and writes one two-digit hex byte per line to ‘image.mem’.

# 3. Module Descriptions

## 3.1 Pixel Enable Generator - pixel_clk_en

Rather than deriving a separate 25 MHz clock , this design keeps everything in the 100 MHz domain and instead generates a single-cycle enable pulse once every four clocks:

```
reg [1:0] div;
always @(posedge clk) div <= rst ? 2'd0 : div + 2'd1;
assign pixel_en = (div == 2'd0);
```

This avoids clock-domain-crossing concerns entirely : there is only ever one clock in the design, at the cost of the VGA counters and pixel output registers only updating on 1 clock in 4, which is functionally equivalent to a 25 MHz pixel clock for timing purposes.

## 3.2 VGA Timing Generator - vga_timing

A single module combines the horizontal and vertical counters. Both counters advance only when pixel_en is high, and both reset synchronously. Standard 640×480 @ 60 Hz timing parameters are used:

| Parameter | Horizontal (pixels) | Vertical (lines) |
|---|---|---|
| Active display | 640 | 480 |
| Front porch | 16 | 10 |
| Sync pulse | 96 | 2 |
| Back porch | 48 | 33 |
| Total | 800 | 525 |

hsync and vsync are both active-low, generated combinationally from hcount/vcount, and video_on is asserted whenever hcount < 640 and vcount < 480.

## 3.3 Source Image ROM - image_rom

A single generic BRAM-inferred ROM stores the source image as 8-bit grayscale samples. It is fully parameterised by DEPTH, ADDR_W and MEM_FILE, and is loaded at elaboration time via $readmemh. The read port is synchronous, which the top level accounts for by delaying valid_in_d by one cycle relative to the address counter.

## 3.4 Sobel Core - sobel_core

Sobel_core internally maintains three full-row shift-register arrays (r1, r2, r3), each WIDTH pixels deep, indexed by the current column counter. On every valid input pixel, the rows shift down (r3 ← r2 ← r1 ← new pixel) and a pair of one-cycle delay taps per row build up the horizontal context needed for a 3×3 neighbourhood.

### Gradient computation

Gx and Gy are computed using the standard Sobel kernels, with all arithmetic carried out on $signed integers to correctly represent negative partial sums:

```
Gx = (r3_now - r3_t2) + ((r2_now - r2_t2) <<< 1) + (r1_now - r1_t2)
Gy = (r3_now + (r3_t1 <<< 1) + r3_t2) - (r1_now + (r1_t1 <<< 1) + r1_t2)
G  = |Gx| + |Gy|                    // Manhattan magnitude approximation
pixel_out = (G > 255) ? 255 : G[7:0]  // saturate to 8 bits
```

The magnitude is saturated to 255 rather than being thresholded , the output is always the full 8-bit gradient magnitude image.

### Window validity

valid_out is only asserted once row > 2 and col > 1,  once three full rows and at least two columns of history have been shifted into the row buffers. Pixels before that point in the raster produce pixel_out = 0 with valid_out = 0, so those output_bram locations are simply never written and keep their reset value of zero, the top and left edges of the resulting edge map are black, while the bottom and right edges are computed normally since the window only looks backwards in row and column, not forwards.

## 3.5 Output Frame Buffer - output_bram

output_bram is a simple dual-port-style BRAM: sobel_core writes computed pixels at waddr = row_out × IMG_WIDTH + col_out whenever valid_out is high, while the VGA display logic independently reads from raddr on every clock. The memory contents are explicitly initialised to zero at simulation time, which is what guarantees a black screen prior to compute_done.

Using a dedicated frame buffer in this way is what allows the Sobel computation to run once, at full clock rate, completely decoupled from the video refresh rate , the trade-off is the extra BRAM tile(s) needed to hold a full copy of the processed frame, on top of the source image ROM.

### 3.6 Top-Level Integration - top_sobel_vga

The top level ties the above modules together :

- addr_cnt / feeding: on reset, addr_cnt starts at 0 and feeding is held high; every clock (full 100 MHz rate, not gated by pixel_en) addr_cnt increments and is fed to image_rom, until it reaches DEPTH−1, at which point feeding drops low.
- valid_in_d: a one-cycle-delayed copy of feeding, used as sobel_core's valid_in, so it stays aligned with the registered ROM output.
- compute_done: raised once feeding and valid_in_d have both gone low, i.e. once the last pixel has fully drained out of sobel_core's pipeline.
- Image placement: the processed image (IMG_WIDTH × IMG_HEIGHT = 200 × 300 in the current parameterisation) is displayed at native resolution — there is no 2× pixel replication or scaling , and is centred on the 640×480 screen using OFFSET_X = (640−IMG_WIDTH)/2 and OFFSET_Y = (480−IMG_HEIGHT)/2.
- Colour output: image_active gates the display to the centred image region and only once compute_done is set; the top nibble of the 8-bit gradient magnitude (rdata[7:4]) is replicated identically onto the 4-bit vga_r, vga_g and vga_b outputs, producing a grayscale edge map. There is no colour pass-through mode and no slide-switch-driven mode selection in this design , the XDC file constrains only clk100, btnC and the VGA pins.
- Reset: btnC is used directly as a synchronous, active-high reset for every always block in the design, consistent with a fully synchronous design style.

## 4. Verification Methodology

A simulation testbench, tb_top_sim, holds reset for three clock cycles, releases it, and then waits on the compute_done flag before dumping the entire contents of output_bram.mem to a hex memory file (output_full.mem) using $writememh. Per the header comment in the testbench, this file is intended to be converted to a viewable image (e.g. PNG) with the companion script ‘mem_to_image.py’ so the computed edge map can be inspected visually and compared against a software reference with the physical board  VGA capture.

Test Image 1 :



Figure 1: Original Image 1



Figure 2 : Testbench Output




Figure 3 : FPGA Output

(Please note that the Green tint was already in the monitor ( Damaged Monitor))

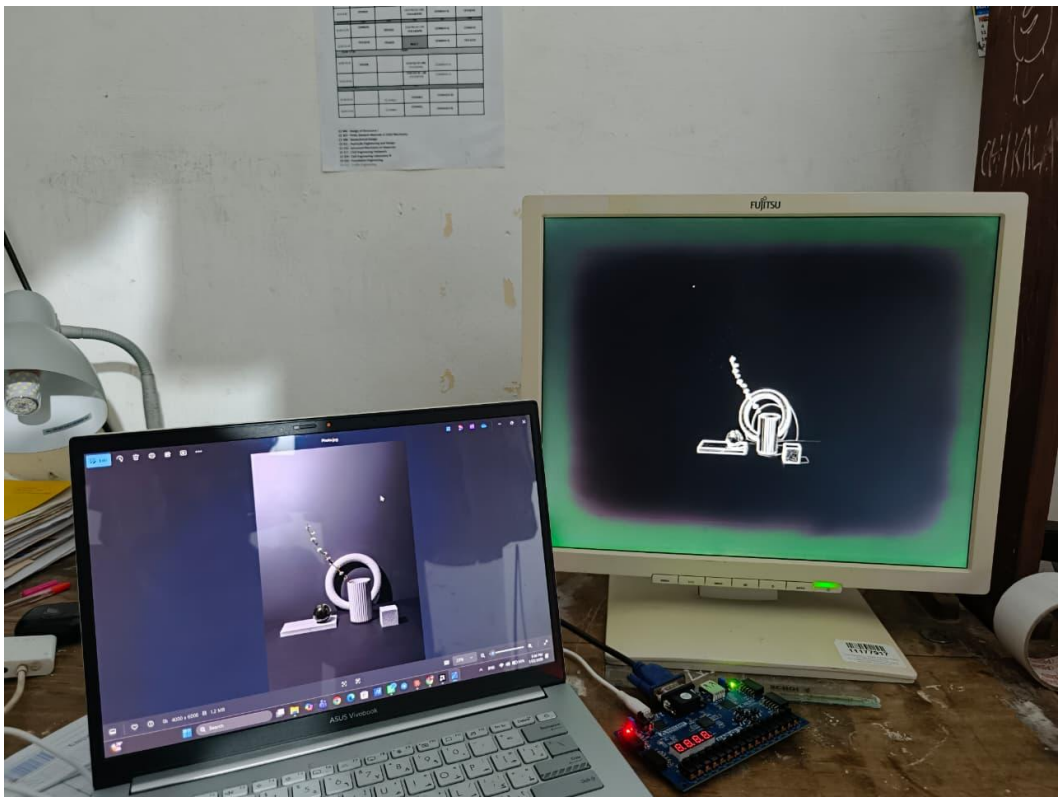



Figure 4 : Comparison of Original image side by side with FPGA output

Test Image 2:



Figure 5: Original Image 2



Figure 6 : Testbench Output




Figure 7 : FPGA Output

Along with that I have also Uploaded the testing video in the following drive link:

https://drive.google.com/drive/folders/1YHIq203xk-0_vTFQreYnRnC40-dqStGK?usp=sharing

# 5. Design Notes and Trade-offs

## 5.1 Single clock domain vs. divided clock

Using an enable strobe instead of a genuinely divided 25 MHz clock avoids clock-domain-crossing hazards and keeps timing closure analysis simple, since every register in the design is clocked by the same 100 MHz source. The cost is that all VGA-side logic must explicitly check pixel_en before updating, rather than relying on the clock edge itself to pace things.

## 5.2 Frame-buffered vs. streaming display

Because the design caches the full computed edge map in output_bram rather than recomputing it continuously in step with the video raster, the Sobel pipeline only needs to run once per image (or once per reset), rather than needing to sustain 25 MHz throughput indefinitely. This significantly relaxes the timing pressure on sobel_core, at the cost of an additional BRAM the size of the whole processed frame. It also means the design as-is only ever displays a single static image; extending it to continuously refresh from a changing source would require re-running the compute pass and swapping frame buffers rather than writing straight into the buffer currently being scanned out.

## 5.3 Fixed-function output

Unlike designs that expose a colour/grayscale/edge mode switch and a runtime-adjustable threshold on slide switches, this implementation has a single, fixed data path: the source image is grayscale on the way in, and the displayed image is always the saturated Sobel gradient magnitude.

## 5.4 Saturating arithmetic

Gx, Gy and the combined magnitude G are computed as plain Verilog integer variables, which is simple to reason about and avoids overflow within sobel_core itself; the final saturation to 8 bits happens only at the very last step before pixel_out is registered.

# 6. Conclusion

The system implements a working Sobel edge detection pipeline for the Basys3 FPGA that separates image computation from VGA display via an intermediate frame buffer, all within a single 100 MHz clock domain. The source image is streamed once through a 3×3 gradient pipeline built directly into sobel_core, saturated to an 8-bit magnitude, cached in output_bram, and then continuously scanned out to a centred region of a standard 640×480 @ 60 Hz VGA display. A dedicated simulation testbench allows the compute path to be verified independently of VGA timing by dumping the resulting frame buffer for offline inspection.

Following are the codes used for the project

## top_sobel_vga.v

```
// top_sobel_vga.v
// Milestone 2 top level.
//

`timescale 1ns / 1ps

module top_sobel_vga #(
   parameter IMG_WIDTH  = 200,
   parameter IMG_HEIGHT = 300,
   parameter MEM_FILE   =
"image.mem"
)(
   input       clk100,
   input       btnC,
   output      vga_hsync,
   output      vga_vsync,
   output [3:0] vga_r,
   output [3:0] vga_g,
   output [3:0] vga_b
);

   localparam DEPTH  = IMG_WIDTH
* IMG_HEIGHT;
   localparam ADDR_W =
$clog2(DEPTH);

   wire rst = btnC;

   wire pixel_en;
   pixel_clk_en u_pixel_en (
      .clk     (clk100),
      .rst     (rst),
      .pixel_en (pixel_en)
   );


   reg [ADDR_W-1:0] addr_cnt;
   reg              feeding;
   reg              valid_in_d;
   reg              compute_done;

   wire [7:0] rom_dout;

   image_rom #(
      .DEPTH    (DEPTH),
      .ADDR_W   (ADDR_W),
      .MEM_FILE (MEM_FILE)
   ) u_image_rom (
      .clk  (clk100),
      .addr (addr_cnt),
      .dout (rom_dout)
   );

   always @(posedge clk100) begin
      if (rst) begin
         addr_cnt     <= 0;
         feeding      <= 1'b1;
         valid_in_d   <= 1'b0;
         compute_done <= 1'b0;
      end
      else begin
         valid_in_d <= feeding;

         if (feeding) begin
            if (addr_cnt == DEPTH-1)
begin
               feeding <= 1'b0;

            end
            else begin
               addr_cnt <= addr_cnt +
1'b1;
            end
         end

         if (!feeding && !valid_in_d)
            compute_done <= 1'b1;
      end
   end

   wire [7:0]  sobel_pixel_out;
   wire        sobel_valid_out;
   wire [31:0] sobel_row_out;
   wire [31:0] sobel_col_out;

   sobel_core #(
      .WIDTH (IMG_WIDTH)
   ) u_sobel_core (
      .clk       (clk100),
      .rst       (rst),
      .pixel_in  (rom_dout),
      .valid_in  (valid_in_d),
      .pixel_out (sobel_pixel_out),
      .valid_out (sobel_valid_out),
      .row_out   (sobel_row_out),
      .col_out   (sobel_col_out)
   );


   wire [ADDR_W-1:0] waddr =
sobel_row_out * IMG_WIDTH +
sobel_col_out;
   wire [ADDR_W-1:0] raddr;
   wire [7:0]        rdata;

   output_bram #(
      .DEPTH  (DEPTH),
      .ADDR_W (ADDR_W)
   ) u_output_bram (
      .clk    (clk100),
      .we     (sobel_valid_out),
      .waddr  (waddr),
      .din    (sobel_pixel_out),
      .raddr  (raddr),
      .dout   (rdata)
   );


   wire [9:0] hcount, vcount;
   wire       video_on;

   vga_timing u_vga_timing (
      .clk      (clk100),
      .rst      (rst),
      .pixel_en (pixel_en),
      .hcount   (hcount),
      .vcount   (vcount),
      .hsync    (vga_hsync),
      .vsync    (vga_vsync),
      .video_on (video_on)
   );


   localparam [9:0] OFFSET_X = (640
- IMG_WIDTH)  / 2;
   localparam [9:0] OFFSET_Y = (480
- IMG_HEIGHT) / 2;

   wire in_image_x = (hcount >=
OFFSET_X) && (hcount <
OFFSET_X + IMG_WIDTH);
   wire in_image_y = (vcount >=
OFFSET_Y) && (vcount <
OFFSET_Y + IMG_HEIGHT);
   wire image_active = video_on &&
compute_done && in_image_x &&
in_image_y;

   assign raddr = image_active
            ? ((vcount - OFFSET_Y) *
IMG_WIDTH + (hcount -
OFFSET_X))
            : {ADDR_W{1'b0}};



   reg [3:0] r_reg, g_reg, b_reg;

   always @(posedge clk100) begin
      if (pixel_en) begin
         if (image_active) begin
            r_reg <= rdata[7:4];
            g_reg <= rdata[7:4];
            b_reg <= rdata[7:4];
         end
         else begin
            r_reg <= 4'd0;
            g_reg <= 4'd0;
            b_reg <= 4'd0;
         end
      end
   end

   assign vga_r = video_on ? r_reg :
4'd0;
   assign vga_g = video_on ? g_reg :
4'd0;
   assign vga_b = video_on ? b_reg :
4'd0;
endmodule
```

## pixel_clk_en.v

```
`timescale 1ns / 1ps

module pixel_clk_en (
    input       clk,
    input       rst,
    output      pixel_en
);

    reg [1:0] div;

    always @(posedge clk) begin
        if (rst)
            div <= 2'd0;
        else
            div <= div + 2'd1;
    end

    assign pixel_en = (div == 2'd0);

endmodule
```

## image_rom.v

```
`timescale 1ns / 1ps

module image_rom #(
    parameter DEPTH    = 512*512,
    parameter ADDR_W   = 19,
    parameter MEM_FILE = "image.mem"
)(
    input                   clk,
    input     [ADDR_W-1:0]  addr,
    output reg [7:0]        dout
);

    reg [7:0] mem [0:DEPTH-1];

    initial begin
        $readmemh(MEM_FILE, mem);
    end

    always @(posedge clk) begin
        dout <= mem[addr];
    end

endmodule
```

## sobel_core.v

```
`timescale 1ns / 1ps

module sobel_core #(
    parameter WIDTH = 512
)(
    input               clk,
    input               rst,
    input     [7:0]     pixel_in,
    input               valid_in,
    output reg [7:0]    pixel_out,
    output reg          valid_out,
    output reg [31:0]   row_out,
    output reg [31:0]   col_out
);

    reg [7:0] r1 [0:WIDTH-1];
    reg [7:0] r2 [0:WIDTH-1];
    reg [7:0] r3 [0:WIDTH-1];

    reg [31:0] col;
    reg [31:0] row;

    wire [7:0] r1_now = r1[col];
    wire [7:0] r2_now = r2[col];
    wire [7:0] r3_now = r3[col];

    reg [7:0] r1_t1, r1_t2;
    reg [7:0] r2_t1, r2_t2;
    reg [7:0] r3_t1, r3_t2;

    integer Gx, Gy, G;

    always @(posedge clk) begin
        if (rst) begin
```

```
            col      <= 0;
            row      <= 0;
            pixel_out <= 8'd0;
            valid_out <= 1'b0;
            row_out  <= 0;
            col_out  <= 0;
            r1_t1 <= 0; r1_t2 <= 0;
            r2_t1 <= 0; r2_t2 <= 0;
            r3_t1 <= 0; r3_t2 <= 0;
        end
        else if (valid_in) begin
            r3[col] <= r2[col];
            r2[col] <= r1[col];
            r1[col] <= pixel_in;

            r1_t2 <= r1_t1; r1_t1 <=
r1_now;
            r2_t2 <= r2_t1; r2_t1 <=
r2_now;
            r3_t2 <= r3_t1; r3_t1 <=
r3_now;

            row_out <= row;
            col_out <= col;

            if (row > 2 && col > 1) begin
                Gx = ($signed({1'b0,
r3_now}) - $signed({1'b0, r3_t2}))
                   + (($signed({1'b0,
r2_now}) - $signed({1'b0, r2_t2})) <<<
1)
                   + ($signed({1'b0, r1_now})
- $signed({1'b0, r1_t2}));
```

```
                Gy = ($signed({1'b0,
r3_now}) + ($signed({1'b0, r3_t1})
<<< 1) + $signed({1'b0, r3_t2}))
                   - ($signed({1'b0, r1_now})
+ ($signed({1'b0, r1_t1}) <<< 1) +
$signed({1'b0, r1_t2}));

                if (Gx < 0) Gx = -Gx;
                if (Gy < 0) Gy = -Gy;
                G = Gx + Gy;

                pixel_out <= (G > 255) ?
8'd255 : G[7:0];
                valid_out <= 1'b1;
            end
            else begin
                pixel_out <= 8'd0;
                valid_out <= 1'b0;
            end

            if (col == WIDTH-1) begin
                col <= 0;
                row <= row + 1;
            end
            else begin
                col <= col + 1;
            end
        end
        else begin
            valid_out <= 1'b0;
        end
    end

endmodule
```

## output_bram.v

```
`timescale 1ns / 1ps

module output_bram #(
    parameter DEPTH  = 512*512,
    parameter ADDR_W = 19
)(
    input                  clk,

    input                  we,
    input      [ADDR_W-1:0]  waddr,
    input      [7:0]         din,

    input      [ADDR_W-1:0]  raddr,
    output reg [7:0]         dout
);
```

```
    reg [7:0] mem [0:DEPTH-1];

    integer i;
    initial begin
        for (i = 0; i < DEPTH; i = i + 1)
            mem[i] = 8'd0;
    end

    always @(posedge clk) begin
        if (we)
            mem[waddr] <= din;
        dout <= mem[raddr];
    end

endmodule
```

## vga_timing.v

```
`timescale 1ns / 1ps

module vga_timing (
    input            clk,
    input            rst,
    input            pixel_en,
    output reg [9:0]  hcount,
    output reg [9:0]  vcount,
    output           hsync,
    output           vsync,
    output           video_on
);

    localparam H_DISPLAY = 640;
    localparam H_FRONT   = 16;
    localparam H_SYNC    = 96;
    localparam H_BACK    = 48;
    localparam  H_TOTAL      = H_DISPLAY + H_FRONT +
H_SYNC + H_BACK;

    localparam V_DISPLAY = 480;
    localparam V_FRONT   = 10;
    localparam V_SYNC    = 2;
    localparam V_BACK    = 33;
    localparam  V_TOTAL      = V_DISPLAY + V_FRONT +
V_SYNC + V_BACK;

    always @(posedge clk) begin
        if (rst) begin
            hcount <= 0;
            vcount <= 0;
        end
        else if (pixel_en) begin
            if (hcount == H_TOTAL-1) begin
                hcount <= 0;
                vcount <= (vcount == V_TOTAL-1) ? 0 : vcount + 1;
            end
            else begin
                hcount <= hcount + 1;
            end
        end
```

```
    end

    assign hsync    = ~(hcount >= H_DISPLAY + H_FRONT &&
hcount < H_DISPLAY + H_FRONT + H_SYNC);
    assign vsync    = ~(vcount >= V_DISPLAY + V_FRONT &&
vcount < V_DISPLAY + V_FRONT + V_SYNC);
    assign  video_on  =  (hcount  <  H_DISPLAY)  &&  (vcount  <
V_DISPLAY);

endmodule
```

## tb_top_sim

```
`timescale 1ns / 1ps

module tb_top_sim;

    localparam IMG_WIDTH  = 200;
    localparam IMG_HEIGHT = 300;
    localparam DEPTH      = IMG_WIDTH * IMG_HEIGHT;
    localparam ADDR_W     = $clog2(DEPTH);
    localparam MEM_FILE   = "image.mem";
    localparam OUT_FILE   = "output_full.mem";

    reg clk = 0;
    reg rst;

    always #5 clk = ~clk; // 100 MHz

    // ---- compute path, mirrors top_sobel_vga.v minus the VGA side ----
    reg  [ADDR_W-1:0] addr_cnt;
    reg               feeding;
    reg               valid_in_d;
    reg               compute_done;

    wire [7:0] rom_dout;

    image_rom #(
        .DEPTH    (DEPTH),
        .ADDR_W   (ADDR_W),
        .MEM_FILE (MEM_FILE)
    ) u_image_rom (
        .clk  (clk),
        .addr (addr_cnt),
        .dout (rom_dout)
    );

    always @(posedge clk) begin
        if (rst) begin
            addr_cnt     <= 0;
            feeding      <= 1'b1;
            valid_in_d   <= 1'b0;
            compute_done <= 1'b0;
        end
        else begin
            valid_in_d <= feeding;
            if (feeding) begin
                if (addr_cnt == DEPTH-1)
                    feeding <= 1'b0;
                else
                    addr_cnt <= addr_cnt + 1'b1;
            end
            if (!feeding && !valid_in_d)
                compute_done <= 1'b1;
        end
    end

    wire [7:0]  sobel_pixel_out;
    wire        sobel_valid_out;
    wire [31:0] sobel_row_out;
    wire [31:0] sobel_col_out;

    sobel_core #(
        .WIDTH (IMG_WIDTH)
    ) u_sobel_core (
        .clk       (clk),
        .rst       (rst),
        .pixel_in  (rom_dout),
        .valid_in  (valid_in_d),
        .pixel_out (sobel_pixel_out),
        .valid_out (sobel_valid_out),
        .row_out   (sobel_row_out),
        .col_out   (sobel_col_out)
    );
```

```
wire [ADDR_W-1:0] waddr = sobel_row_out * IMG_WIDTH + sobel_col_out;

    output_bram #(
        .DEPTH  (DEPTH),
        .ADDR_W (ADDR_W)
    ) u_output_bram (
        .clk    (clk),
        .we     (sobel_valid_out),
        .waddr  (waddr),
        .din    (sobel_pixel_out),
        .raddr  ({ADDR_W{1'b0}}),
        .dout   ()
    );

    initial begin
        rst = 1;
        repeat (3) @(posedge clk);
        rst = 0;

        wait (compute_done == 1'b1);
        repeat (5) @(posedge clk);

        $writememh(OUT_FILE, u_output_bram.mem);
        $display("Wrote frame buffer contents to %s (%0dx%0d)", OUT_FILE, IMG_WIDTH, IMG_HEIGHT);
        $finish;
    end

endmodule
```

## Constraint File code

```
## basys3_vga.xdc
## Clock, reset button, and VGA pin constraints for the Basys3 board.
## Pin numbers are from Digilent's standard Basys3 master XDC.

## Clock signal (100 MHz)
set_property PACKAGE_PIN W5 [get_ports clk100]
set_property IOSTANDARD LVCMOS33 [get_ports clk100]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk100]

## Center button (used as reset)
set_property PACKAGE_PIN U18 [get_ports btnC]
set_property IOSTANDARD LVCMOS33 [get_ports btnC]

## VGA Connector
set_property PACKAGE_PIN G19 [get_ports {vga_r[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vga_r[0]}]
set_property PACKAGE_PIN H19 [get_ports {vga_r[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vga_r[1]}]
set_property PACKAGE_PIN J19 [get_ports {vga_r[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vga_r[2]}]
set_property PACKAGE_PIN N19 [get_ports {vga_r[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vga_r[3]}]

set_property PACKAGE_PIN N18 [get_ports {vga_g[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vga_g[0]}]
set_property PACKAGE_PIN L18 [get_ports {vga_g[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vga_g[1]}]
set_property PACKAGE_PIN K18 [get_ports {vga_g[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vga_g[2]}]
set_property PACKAGE_PIN J18 [get_ports {vga_g[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vga_g[3]}]

set_property PACKAGE_PIN J17 [get_ports {vga_b[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vga_b[0]}]
set_property PACKAGE_PIN H17 [get_ports {vga_b[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vga_b[1]}]
set_property PACKAGE_PIN G17 [get_ports {vga_b[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vga_b[2]}]
set_property PACKAGE_PIN D17 [get_ports {vga_b[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {vga_b[3]}]

set_property PACKAGE_PIN P19 [get_ports vga_hsync]
set_property IOSTANDARD LVCMOS33 [get_ports vga_hsync]
set_property PACKAGE_PIN R19 [get_ports vga_vsync]
set_property IOSTANDARD LVCMOS33 [get_ports vga_vsync]
```

## image_to_mem.py

```python
import sys
from PIL import Image


def main():
    if len(sys.argv) < 3:
        print("Usage: python3 image_to_mem.py <input_image> <image.mem> [max_width] [max_height]")
        sys.exit(1)

    in_path = sys.argv[1]
    out_path = sys.argv[2]
    max_w = int(sys.argv[3]) if len(sys.argv) > 3 else 200
    max_h = int(sys.argv[4]) if len(sys.argv) > 4 else 300

    img = Image.open(in_path).convert('L')
    w, h = img.size
    scale = min(max_w / w, max_h / h, 1.0)
    if scale != 1.0:
        w, h = int(w * scale), int(h * scale)
        img = img.resize((w, h), Image.LANCZOS)

    pixels = list(img.getdata())

    with open(out_path, 'w') as f:
        for p in pixels:
            f.write('%02x\n' % p)

    print(f"Wrote {len(pixels)} pixels ({w}x{h}) to {out_path}")
    print("Set these in top_sobel_vga.v (or as Vivado generic overrides):")
    print(f"  IMG_WIDTH  = {w}")
    print(f"  IMG_HEIGHT = {h}")
    if w > 640 or h > 480:
        print("WARNING: image is larger than 640x480 and will not fit on screen.")


if __name__ == '__main__':
    main()
```

## mem_to_image.py

```
import sys
from PIL import Image


def main():
    if len(sys.argv) < 5:
        print("Usage: python3 mem_to_image.py
<output_full.mem> <width> <height>
<result.png>")
        sys.exit(1)

    mem_path = sys.argv[1]
    w = int(sys.argv[2])
    h = int(sys.argv[3])
    out_png = sys.argv[4]

    with open(mem_path) as f:
        values = [int(line.strip(), 16) for
line in f if line.strip()]

    expected = w * h
    if len(values) != expected:
        print(f"WARNING: expected {expected}
pixels ({w}x{h}), got {len(values)}.")

    img = Image.new('L', (w, h), color=0)
    px = img.load()
    idx = 0
    for r in range(h):
        for c in range(w):
            if idx >= len(values):
                break
            px[c, r] = values[idx]
            idx += 1

    img.save(out_png)
    print(f"Saved {out_png} ({w}x{h})")


if __name__ == '__main__':
    main()
```